

Roadmap & Specifications

Two-Wheeler Post-Crash Detection System

1. Validated Component List & Connectivity

All listed components have been verified for compatibility with the ESP32 architecture. The ESP32 is a powerhouse that can handle multiple communication protocols simultaneously

Component	Exact Model/Name	Communication Protocol	Notes
Microcontroller	ESP32 DevKit V1 (30 or 38 pin)	USB (Programming)	The brain of the system. Has 520KB SRAM, enough for TinyML inference.
Bluetooth / BLE	Integrated in ESP32	Internal	<i>Do not buy a separate Bluetooth module (like HC-05).</i> The ESP32 chip has a built-in Bluetooth Low Energy (BLE) radio, which you will use to communicate with the Companion App.
IMU Sensor	GY-521 MPU6050	I2C (Pins: SDA, SCL)	6-axis Accelerometer & Gyroscope. Has a dedicated interrupt pin (INT) crucial for waking the ESP32 from Deep Sleep.
GPS Module	U-blox NEO-6M	UART (Hardware Serial)	Provides latitude/longitude. Connect to ESP32 Hardware UART1 (e.g., pins 13/15).
GSM Module	SIM800L EVB (or V2.0)	UART (Hardware Serial)	Sends the emergency SMS. Connect to ESP32 Hardware UART2 (e.g., pins 16/17). Ensure your local network still supports 2G.
Voice Module	DFRobot DFPlayer Mini	UART (Software Serial)	Plays MP3 voice prompts. Connect via EspSoftwareSerial library to any two standard GPIO pins. Requires a MicroSD

Component	Exact Model/Name	Communication Protocol	Notes
			card (FAT32, <32GB) to store the .mp3 files.
Speaker	3W 8Ohm Mini Speaker	Direct Audio Out	Wires directly to the SPK_1 and SPK_2 pins on the DFPlayer Mini.
User Input	Standard Tactile Button	GPIO (Digital Read)	Used by the rider to cancel the 30-second alert countdown.
Power Supply	18650 Li-ion Cell (3.7V)	DC Power	Minimum 2000mAh capacity recommended.
Charge/Protect	TP4056 Module	DC Power	Safely charges the 18650 battery via USB-C or Micro-USB.
Boost Converter	MT3608 Boost Module	DC Power	Steps up the 3.7V battery voltage to 5V to power the ESP32 and SIM800L (SIM800L can draw up to 2A bursts, so ensure steady power).

System Connectivity & Feasibility

Everything can communicate simultaneously.

- **I2C Bus:** Handles the MPU6050.
- **UART0:** Reserved for USB programming/debugging.
- **UART1 & UART2:** Hardware serial ports used for GPS and GSM (high priority, heavy data).
- **SoftwareSerial:** Used for the DFPlayer Mini (low priority, simple commands).
- **Internal Radio:** Handles the BLE connection to the app without needing extra pins.

2. Immediate Action Plan

While waiting for your hardware delivery, you can complete the majority of the software architecture.

Step 1: Secure Datasets & Understand the Data

- **Action:** Go to Google Dataset Search, Kaggle, or Mendeley Data.
- **Search Queries:** "Motorcycle Fall Detection Dataset", "Smartphone accelerometer dataset for crash".
- **Goal:** Download the datasets (usually .csv files). Open them in Excel or Google Sheets. You will see columns for Time, Accel_X, Accel_Y, Accel_Z, Gyro_X, etc. Familiarize yourself with what a spike in G-force looks like in raw numbers.

Step 2: Begin Machine Learning Prep

- **Action:** Create a free account on **Edge Impulse** (edgeimpulse.com). This is the best platform for TinyML on ESP32.
- **Task:** Create a new project. Use the "Data Acquisition" tab to upload the .csv files you downloaded.
- **Task:** Create an "Impulse" (your ML pipeline). Set it up to take Time-Series data, pass it through a "Spectral Analysis" block (which automatically extracts features like peak frequencies and energy), and then route it into a "Classification" block (Neural Network or Random Forest).
- **Goal:** Train a basic model entirely in your browser using the public data.

Step 3: Build the Companion App (Frontend)

- **Action:** Choose an app framework. **Flutter** or **React Native** are standard, but if you want something fast and visual, use **MIT App Inventor** or **FlutterFlow**.
- **Task:** You do not need the ESP32 to build the user interface. Build the following screens:
 1. **Splash Screen / Logo.**
 2. **Dashboard:** Displaying mock data (e.g., "Safety Score", "Potholes hit").
 3. **Settings / Contacts Page:** Text input fields to save 3 emergency contact phone numbers.
 4. **Pairing Screen:** A simple UI button that says "Connect to Helmet/Bike".

Step 4: Record Voice Prompts

- **Action:** Use an AI voice generator (like ElevenLabs or similar free tools) to generate professional, calm voice prompts.
- **Task:** Generate and save these as .mp3 files (name them 0001.mp3, 0002.mp3, etc., as required by the DFPlayer file system):
 - *"System Armed. Ride safely."*
 - *"Impact detected. Alerting emergency contacts in 30 seconds. Press the button to cancel."*
 - *"Alert cancelled."*
 - *"Emergency SMS sent."*

3. Project Implementation Roadmap

Phase 1: Hardware Assembly & Basic Validation (Weeks 1-2)

- **Objective:** Ensure all bought components work individually before merging code.
- **Tasks:**
 - Solder/Breadboard the components according to the power constraints (e.g., ensuring SIM800L gets enough current).
 - Write a script to test MPU6050 serial output.
 - Write a script to verify the GPS gets a satellite lock.
 - Write a script to send a hardcoded SMS via SIM800L.
 - Play the .mp3 files via the DFPlayer Mini.

Phase 2: Custom Data Collection & ML Finalization (Weeks 3-4)

- **Objective:** Train a model specific to your hardware mounting.
- **Tasks:**
 - Write an ESP32 script that just reads MPU6050 data and outputs it to the Serial Monitor.
 - Use a Python script (or Edge Impulse CLI) on your laptop to capture this serial data into .csv files while you simulate drops, hard braking, and hitting bumps with the device.
 - Merge this custom data with your public dataset on Edge Impulse.
 - Train the final model and export it as an **Arduino Library**.

Phase 3: The "Brain" Integration (Weeks 5-6)

- **Objective:** Combine the ML model with the hardware logic.
- **Tasks:**
 - Import the Edge Impulse Arduino Library into your ESP32 IDE.
 - Write the core pipeline: Read IMU -> Buffer 3s Data -> Pass to ML Model -> Get Result.
 - **State Machine:** Implement the logic for the "Crash" result. If Crash -> Play voice prompt -> Start 30s countdown -> Wait for button press. If no press -> Get GPS coords -> Send SMS.

Phase 4: App & Ecosystem Integration (Weeks 7-8)

- **Objective:** Connect the hardware to the phone app.
- **Tasks:**
 - Implement **BLE Server** code on the ESP32.
 - Connect the Companion App to the ESP32.
 - Write code so the app can push the emergency contact numbers to the ESP32 (saving them to the ESP32's non-volatile memory/EEPROM).
 - *(Optional Premium Feature)* Code the app to send a "Cancel Alert" BLE command if the phone detects the user is walking (handling parking lot drops).

Phase 5: Field Testing & Deep Sleep Optimization (Weeks 9-10)

- **Objective:** Make the device battery efficient and test in the real world.
- **Tasks:**
 - Configure the MPU6050 to send an interrupt signal when it detects a high G-force.
 - Configure the ESP32 to stay in Deep Sleep and only wake up when it receives that interrupt.
 - Mount the device on a two-wheeler in a weatherproof enclosure.
 - Perform real-world test rides. Measure the *False Positive Rate* (did hitting a speed breaker trigger the buzzer?). Tweak the ML model thresholds if necessary.